

RNZ tool guide

Oct 9, 2026 · @Moe

The RNZ tool gives apps built in VS Code the Ricoh New Zealand (RNZ) look, and keeps them that way. It makes new branded Microsoft Fabric apps, and re-skins apps you already have without breaking them. Release 1.2.0.

What it is

Anyone at RNZ who builds apps in VS Code with GitHub Copilot can use it. You type short commands in Copilot Chat, and the tool does the work.

- **New apps** start with the RNZ look already built in: colours, fonts, the RICOH lock-up, ready-made components and a brand check before every deploy.
- **Apps you already have** get the RNZ colours, fonts and font weights. Nothing else changes: no layout, components, logic, wording or file names.
- **Afterwards**, Copilot and Claude Code follow the RNZ rules for that app automatically, so new work stays on brand without repeated corrections.
- **Optional agents** can build, review and record changes for you.

Everything that changes an app happens on a separate git branch. You look, then keep it or throw it away. The tool keeps itself up to date from GitHub, so everyone always has the latest release.

Set up once

Setup takes about five minutes per person, and you only do it once.

You need:

- Access to the `mosafin/rnz-fabric-apps` repository on GitHub (ask the maintainer)
- Git
- Node.js 20 or later
- VS Code with GitHub Copilot, signed in

To check Git and Node, open a terminal in VS Code (Terminal > New Terminal) and run `git --version` and `node --version`. Both should print a version, and Node should be 20 or higher.

Steps:

1. In the VS Code terminal, paste these two lines and press Enter. A GitHub sign-in window may open the first time.

```
git clone https://github.com/mosafin/rnz-fabric-apps "$HOME/.rnz/rnz-fabric-apps"
node "$HOME/.rnz/rnz-fabric-apps/bin/rnz.mjs" install
```

2. Reload VS Code: press `Ctrl + Shift + P`, type `reload window` and press Enter.
3. Open Copilot Chat, choose **Agent** mode and type `/rnz`. Five commands should appear.

The first line makes a folder called `.rnz` in your user folder. That's the tool's own copy: you never need to open or edit it. You never need to install again either, because the tool and its commands update themselves.

The five commands

Type these in Copilot Chat, in **Agent** mode. Each one asks you questions or shows a preview before it changes anything.

Command	Use it to	Changes files?
<code>/rnz-new</code>	Create a new branded Fabric app	Creates a new app folder
<code>/rnz-skin</code>	Put the RNZ look on an app you already have	Yes, on a new branch
<code>/rnz-update</code>	Bring an app up to the latest RNZ release	Yes, on a new branch
<code>/rnz-agents</code>	Add the builder, QA or reporter agents	Adds files, on a new branch
<code>/rnz-check</code>	See what's off brand	No

Every time a command changes an app:

- **Before:** open only the app's repository in VS Code, at its top folder. Commit or stash your work, because the tool stops if anything is uncommitted. If the repository is in OneDrive, pause OneDrive.
- **After:** try the app on the new branch with `npm run dev`. If you're happy, merge the branch. If not, switch back to your own branch and delete the new one. Your branch was never touched.

The same commands work in a terminal without Copilot. Run them in the app's folder (the `.` means "this folder"):

```
$rnz = "$HOME/.rnz/rnz-fabric-apps/bin/rnz.mjs"
node $rnz new my-app --agents builder,qa
node $rnz skin .
node $rnz skin . --apply
node $rnz update . --apply
node $rnz agents . --agents all
node $rnz check .
node $rnz help
```

Make a new app

`/rnz-new` creates a ready-to-build Microsoft Fabric data app with the RNZ look, components, rules and brand check already in place.

1. In VS Code, open Copilot Chat in **Agent** mode and type `/rnz-new`.
2. Answer the questions:
 - **App name:** lowercase letters, numbers and hyphens, for example `pipeline-by-pillar`.
 - **Fabric workspace ID** (optional): the long ID after `/groups/` in the workspace's web address. Say you don't have it if you don't.
 - **Builder, QA and reporter agents:** yes or no to each (see Agents below).
3. The tool creates the app in `C:\dev\APP-NAME`, checks the brand, makes the first commit and opens it in a new VS Code window.
4. In the new window, open Copilot Chat (**Agent** mode). If you added the builder, pick **rnz-builder** from the agent list. Then type: *Build this RNZ app. It's for [who] to see [what]. Model: [semantic model share link]. Pillar: [pillar or none].*

What you get:

- The RNZ theme: colours, Frutiger then Arial, type sizes, chart and table styles.
- A kit of RNZ components, including the app frame with the RICOH lock-up.
- Rules and skills that Copilot and Claude Code follow automatically, covering design, layout, screen wording and a QA checklist.
- A brand check (`npm run rnz:check`) that also runs before every deploy and stops a deploy with brand errors.

The look is fixed. Which screens you build, and what they show, is up to you.

Next:

- **Put it on GitHub:** Source Control > Publish Branch > choose **private**.
- **Deploy:** ask Copilot to deploy, or run `npx rayfin up` in the app folder.
- **Keep it current:** run `/rnz-update` after each RNZ release.
- Keep apps outside OneDrive (`C:\dev` is the default for that reason), and use real data only.

Re-skin an existing app

`/rnz-skin` changes an app's **colours, fonts and font weights** to RNZ, and nothing else. It's built and tested for React and Vite web apps, including Fabric data apps and apps styled with Tailwind or shadcn. Apps made with `/rnz-new` already have the look; use `/rnz-update` for those.

Before you start: open the app's repository in VS Code at its top folder, commit or stash everything (new files count too), and pause OneDrive if the repository is in it. Not sure yet? Type `/rnz-check` first. It lists what the skin would change, and changes nothing.

1. Type `/rnz-skin` in Copilot Chat (**Agent** mode).
2. The tool previews the change, then applies it on a new branch called `rnz-skin`. It runs your build and tests before and after. If anything that passed before now fails, it undoes everything by itself and tells you why.
3. Copilot then swaps the hardcoded colours, fonts and weights in your own files for RNZ values, and commits on the same branch. It only changes values.
4. Try it: run `npm run dev` in the app folder and look through the screens.
5. If you're happy, merge `rnz-skin` into your main branch. If not, switch back and delete `rnz-skin`.

What it adds:

Added	What it's for
<code>src/rnz-skin.css</code>	Maps your app's own colour, font and weight settings to RNZ values
<code>src/rnz-lockup.tsx</code> and the lock-up image	The RICOH lock-up, ready to place

Added	What it's for
<code>rnz/</code> folder	Settings, a check script and a report of hardcoded values
<code>.agents/skills/rnz-skin/</code>	The rules Copilot and Claude Code follow
One line or one marked block in a few files	The skin import, an RNZ block in <code>AGENTS.md</code> , a <code>rnz:skin-check</code> script and Copilot instructions

Good to know:

- **Light theme.** If your app has no dark mode styles, the skin keeps it light. If it does, dark mode is left alone and the report says so.
- **The logo isn't placed for you**, because that would be a layout change. Ask Copilot to add `<RicohLockup />` where you want it.
- **Colours with their own meaning**, such as a set of status colours, are left alone and listed in `rnz/skin-report.md`.

Keep apps up to date

Apps only change when you ask, so run `/rnz-update` on each app after a new RNZ release.

- `/rnz-update` works out whether the app was made with `/rnz-new` or has the skin, and shows a preview. When you confirm, it applies the update on a new branch with the same before-and-after checks as the skin. For a skinned app, it redoes the skin from the latest release.
- `/rnz-check` reports anything off brand: the file, the line and the RNZ value to use instead. It changes nothing. Use it any time, for example before a review or a deploy.

Apps made with `/rnz-new` also run the brand check automatically before every deploy, and a deploy with brand errors stops until they're fixed.

Agents

Three optional helpers that Copilot and Claude Code can use. `/rnz-new` asks whether you want each one; add them to an app later with `/rnz-agents` (an existing app needs the RNZ skin first).

Agent	Does	Never does
<code>rnz-builder</code>	Builds screens and changes following the RNZ rules, then runs the brand check, tests and build	Edit brand files, rename files, commit, push or deploy
<code>rnz-qa</code>	Reviews a change against the brand, accessibility, wording and data rules, and lists the problems	Change any file: it has no editing tools, so the work never checks itself
<code>rnz-reporter</code>	Updates the version number and adds a plain-English change log entry	Change any other file, commit, push or deploy

Use them:

- In **Copilot**, pick the agent from the agent list (where you choose Agent mode), then ask.
- In **Claude Code**, ask for it by name, for example "use `rnz-qa` to review this change".
- Open the repository at its top folder, so the agents can read its rules and change log.
- When QA asks to run `npm` or `git` commands, approve them. It needs them to run the checks, and it still can't change files.

A typical loop: ask **rnz-builder** for a change, then **rnz-qa** to review it, then **rnz-reporter** to record it.

The reporter and your change log: if the app already has one (`VERSIONING.md` , `CHANGELOG.md` , `CHANGES.md` or `HISTORY.md`), the reporter uses it and follows its rules; if not, it starts a `CHANGELOG.md` . A new app with the reporter starts at version 0.1.0, and an existing app keeps its version. By default, a fix moves the last number (0.1.0 to 0.1.1), a new screen or feature moves the middle number (0.1.1 to 0.2.0), and the first number moves only when you say so. If your repository's own rules already make every agent log its changes, you may not need the reporter.

Brand rules

The tool applies these rules for you, and Copilot follows them in new work.

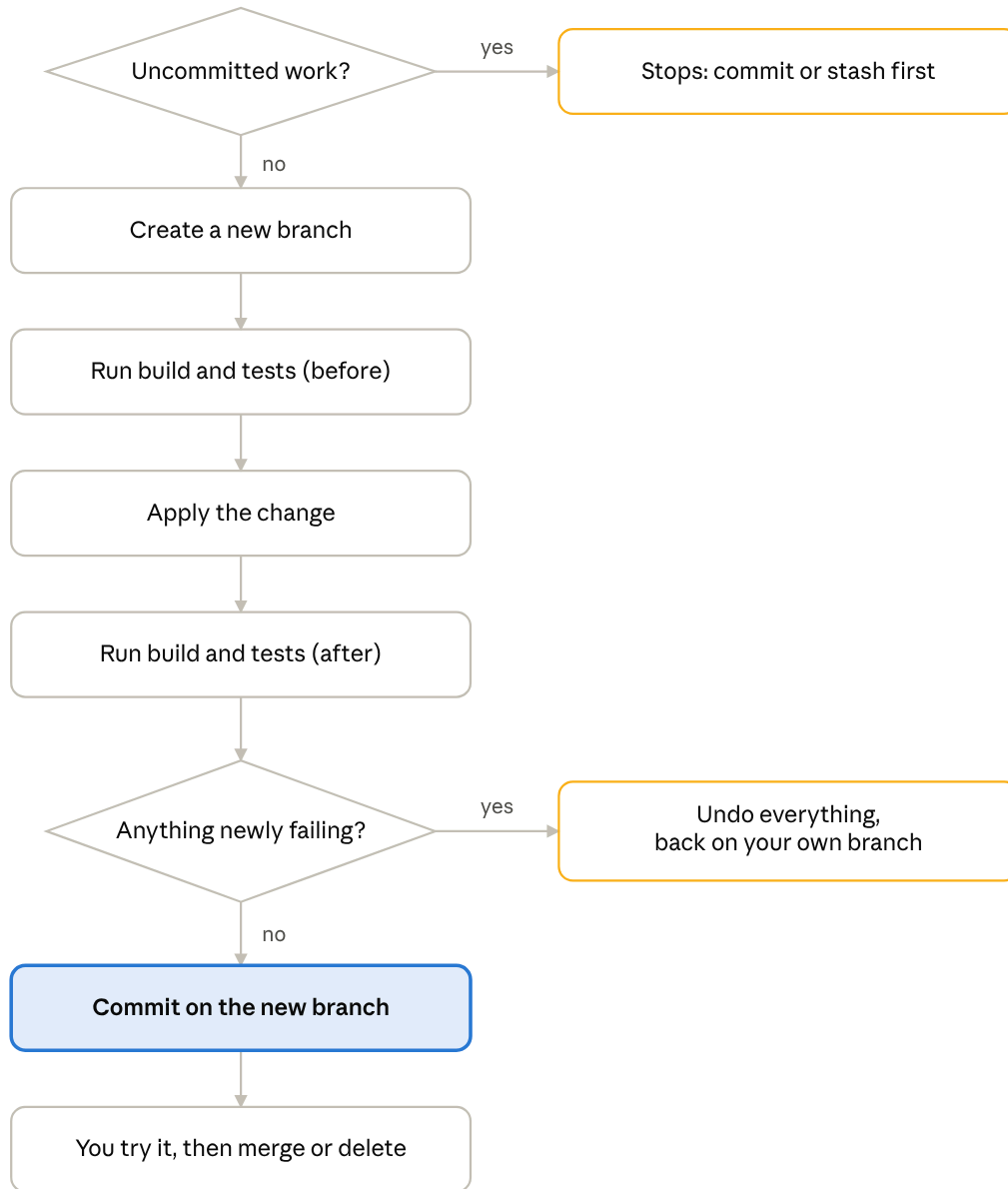
Colour	Value	Used for
Ricoh Red	#CF142B	The main action on a screen and key emphasis, nothing decorative
Ricoh Grey	#666666	All text and icons. Never black or near-black
White	#FFFFFF	Page and card backgrounds
Light grey	#F5F5F5	Panels, table headers, hover rows
Border grey	#E0E0E0	Dividers and card outlines

- **Type:** Frutiger, then Arial. No other fonts and no font links. Weights 300 (large figures), 400 (body) and 700 (headings, labels, buttons) only.
- **Theme:** light only. No dark headers, bands or sidebars. White text only on red or grey.
- **Logo:** the RICOH logo always with its "imagine. change." tagline (the lock-up), at least 80px wide. Never on its own, never cropped or redrawn, never as a favicon. Using only the lock-up in apps is an RNZ decision awaiting APAC brand approval.
- **Words on screens:** NZ English, sentence case, no em or en dashes, no all-capital labels, buttons that say what they do.
- **Charts:** red for the key series, then grey, then the muted blue, teal and green.
- **Data:** real data only. No invented figures.

How it keeps your app safe

A change is kept only if everything that worked before still works, and it never lands on your own branch.

Your branch is never touched; a change stays only if nothing breaks



what the skin and updates do with your app · 7 steps, 2 checks

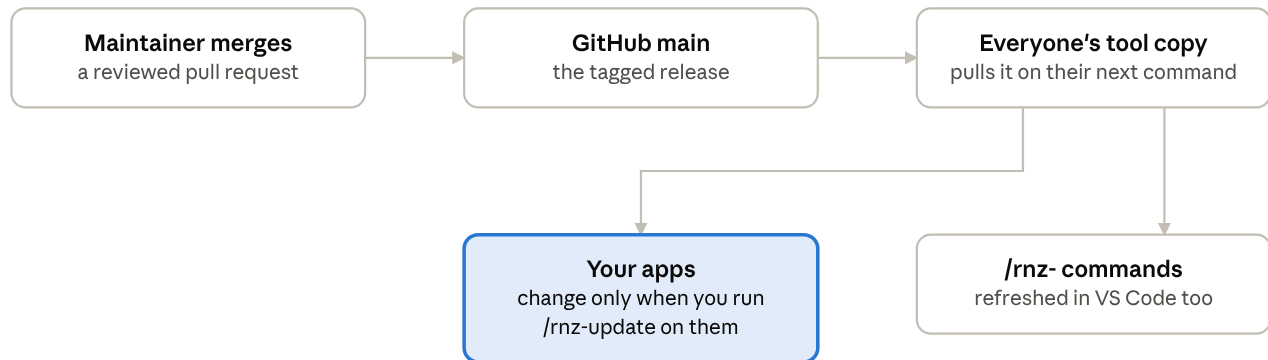
Both exits on the right leave your app exactly as it was. `/rnz-agents` follows the same path but skips the build and tests, because it only adds instruction files.

It never: restructures screens, edits your own CSS or components, renames or deletes your files, pushes, merges or deploys. Those stay with you. In your existing files it adds at most one line or one clearly marked block, and keeps the rest exactly as it was.

How updates reach everyone

Nobody reinstalls: every `/rnz-` command first updates the tool from GitHub, refreshes the commands in VS Code, then does its job.

Releases reach everyone automatically; apps change only on `/rnz-update`



how a release travels · maintainer to everyone's tool

Your apps are the exception on purpose. They only change when you run `/rnz-update` on them, so a release never surprises an app mid-project.

Troubleshooting

Find the message or symptom, then follow the fix. Still stuck? Send the maintainer the command you ran and everything it printed.

You see	Do this
<code>git clone</code> says "Repository not found" or asks for access	You don't have access yet, or you're signed in to the wrong GitHub account. Ask the maintainer to add you.
"Node ... is too old"	Install Node.js 20 or later, close and reopen VS Code, then try again.
The <code>/rnz</code> commands don't show in Copilot Chat	Reload VS Code and check Copilot Chat is in Agent mode. Still missing? Run the install line from Set up once again.
"The repository has uncommitted changes"	Commit your work, or run <code>git stash -u</code> and bring it back later with <code>git stash pop</code> .

You see	Do this
"Deletion of directory ... failed. Should I try again? (y/n)"	Type <code>n</code> and press Enter. OneDrive or the app preview (<code>npm run dev</code>) had the folder open. Pause OneDrive and stop the preview next time.
Copilot ran a command in the wrong repository	VS Code had several repositories open. Open only the app's repository, or run git yourself with the folder named: <code>git -C "C:\path\to\repo" status</code> .
"More than one app found" or "No web app found"	Point the command at the app's own folder, the one with its <code>package.json</code> .
"... doesn't have the RNZ brand layer yet" from <code>/rnz-agents</code>	Run <code>/rnz-skin</code> first. If you already did, check you're on the branch that has it: <code>git branch --show-current</code> .
"... was undone, because something that worked before stopped working"	The safety check worked: nothing changed and you're back on your branch. Send the full message to the maintainer.
"Note: couldn't get the latest RNZ tool"	You're offline, or the tool's own copy was edited. The command still runs. To reset the tool's copy (never your apps), run the two lines below.
The agents don't show in Copilot's agent list	Open the repository at its top folder, then reload VS Code.
QA keeps asking to run commands	Expected. Approve the <code>npm</code> and <code>git</code> commands; it still can't change files.
"... is not digitally signed" or "running scripts is disabled"	That's the older PowerShell scripts. Use the <code>/rnz-</code> commands, or unblock the file with <code>Unblock-File</code> .

To reset the tool's own copy:

```
git -C "$HOME/.rnz/rnz-fabric-apps" fetch
git -C "$HOME/.rnz/rnz-fabric-apps" reset --hard origin/main
```

For maintainers

This part is for whoever looks after the tool. The full rules live in the repository's root `AGENTS.md`.

Give someone access: on GitHub, open the repository's **Settings > Collaborators > Add people** and give read access. For a larger group, move the repository to a GitHub organisation and give a team read access. Then send them this guide.

Make a release:

1. Make the change on a new branch in your own clone, never in the `.rnz` copy.
2. Run the checks. In `templates/rnz-data-app: npm ci, npm run rnz:check, npx vitest run src` and `npm run build:fabric`. From the root: `node scripts/rnz-lock.mjs --verify, node scripts/test-skin.mjs` and `node scripts/test-agents.mjs`.
3. If you changed the template, bump the version (in `rnz/brand-manifest.json, package.json` and `rayfin-template.yml`) and run `node scripts/rnz-lock.mjs`.
4. Add a `CHANGELOG.md` entry.
5. Open a pull request. The Template checks must pass.
6. Merge, then tag the release from the terminal, naming the folder so the tag lands in the right repository (`X.Y.Z` is the new version):

```
git -C "$HOME/.rnz/rnz-fabric-apps" pull
git -C "$HOME/.rnz/rnz-fabric-apps" tag vX.Y.Z
git -C "$HOME/.rnz/rnz-fabric-apps" push origin vX.Y.Z
```

Everyone gets the release the next time they run an `/rnz-` command.

Test a branch before merging: point your tool copy at the branch with `git -C "$HOME/.rnz/rnz-fabric-apps" fetch` then `git -C "$HOME/.rnz/rnz-fabric-apps" checkout BRANCH-NAME`, test with the `/rnz-` commands, then `git -C "$HOME/.rnz/rnz-fabric-apps" checkout main`.

Rules that never change:

- Never rename or move a file. Skills, scripts and other repositories use the exact names.
- The RICOH logo is always the lock-up. Never a logo-only file, a cropped tagline or a favicon.
- The brand guide's source is `ricoh-brand-methodology-v12.html` in the `rnz-ai-library` repository. Change it there first, then copy it into the template unchanged.

- The tool must never need the `rnz-ai-library` repository to run.
- The skin changes values only: colours, fonts and font weights, never structure.
- NZ English, no em or en dashes in anything people read.
- Run release git commands in the terminal with `git -C "folder"`, not through Copilot Chat. With several repositories open, Copilot can pick the wrong one.

Glossary

Plain meanings of the terms this guide uses.

Term	Meaning
Agent mode	The Copilot Chat setting that lets Copilot run commands and change files for you. The <code>/rnz-</code> commands need it.
Agent (rnz-builder, rnz-qa, rnz-reporter)	A helper with a fixed job and fixed limits that Copilot or Claude Code can switch to.
Branch	A separate line of work in a repository. Changes on a branch don't touch your main work until you merge them.
Commit	A saved snapshot of changes in git, with a short message.
Fabric app	A web app that runs inside Microsoft Fabric and shows data from a Power BI semantic model.
Lock-up	The RICOH logo together with its "imagine. change." tagline. The only way the logo appears in RNZ apps.
Merge	Bringing a branch's changes into another branch, usually your main one.
Pull request	A request on GitHub to merge a branch, where checks run and changes can be reviewed first.
Repository (repo)	A project folder tracked by git, usually also stored on GitHub.

Term	Meaning
Skin	The RNZ colours, fonts and font weights applied to an app you already have, without changing anything else.
Stash	Putting uncommitted changes aside temporarily (<code>git stash -u</code>), and bringing them back later (<code>git stash pop</code>).
Tag	A permanent label on a commit that marks a release, such as <code>v1.2.0</code> .
Template	The starting point every new RNZ app is made from.
Terminal	The command window in VS Code (Terminal > New Terminal), where you paste commands.
Token	A named value, such as <code>--rnz-red</code> , used instead of typing a colour or font directly, so it can change in one place.